

Loop-, While- und Goto-Programme

Sebastian Böhne
boehne@uni-potsdam.de



- 1 Vorbetrachtungen
- 2 Loop-Programme
- 3 While-Programme
- 4 Goto-Programme

Worum geht es?

- Alternative Berechenbarkeitsmodelle
- Orientierung an imperativen Programmiersprachen
- Auch Versuch, genau die total berechenbaren Funktionen zu charakterisieren
- Wie mächtig im Vergleich untereinander und mit den TMs?

Orientierung an imperativen Programmiersprachen

- Informatik als Kunst, das Rechnen dem Computer zu überlassen
- TMs wie Hardware, gespickt mit viel zu vielen Details
- Imperativer Programmierstil (nächst)höheres Level
- Guter Programmierstil nicht(!) das Ziel

Totale Berechenbarkeit

- Zuvor Funktionen, die durch TMs berechnet werden, die stets anhalten
- Zumindest a priori nicht feststellbar, für welche TMs das zutrifft (kein äußeres Merkmal)
- Grundproblem: Gleiches Berechenbarkeitsmodell (TMs) für verschiedene Berechenbarkeitsbegriffe (totale Turing-Berechenbarkeit und Turing-Berechenbarkeit)
- Daher: Versuch, der Form nach unterscheidbare Berechenbarkeitsmodelle zu finden

Mächtigkeiten der Berechenbarkeitsmodelle

- Berechenbarkeitsmodelle vergleichbar?
- Überlegenheit eines Modells?
- (Alle) Modelle äquivalent?
- Oder Unvergleichbarkeit wegen unterschiedlicher Stärken?
 - f, g Funktionen, die berechenbar sein sollten?
 - Modell 1 erfasst Funktion f , aber nicht g
 - Modell 2 erfasst g , aber nicht f
 - Was bedeutet es, dass eine Funktion berechenbar sein sollte?

Definition

- Loop-Programme bestehen aus folgenden Komponenten:
 - der Konstanten `0`
 - den Variablen `xi` für $i \in \mathbb{N}$
 - den einstelligen Operatoren `suc` und `pred`
 - dem Zuweisungsoperator `:=`
 - dem Kompositionsoperator `;`
 - dem Schleifenkonstrukt `loop do end`
- $\mathcal{LP} :=$ Menge aller Loop-Programme

Formale Syntax

- $G_n := (V_n, T, P_n, \langle \text{Loop-Programm}_n \rangle)$ mit
- $V_n := \{ \langle \text{Loop-Programm}_n \rangle, \langle \text{Anweisungsblock} \rangle, \langle \text{Elementaranweisung} \rangle, \langle \text{Variable}_n \rangle \}$
- T das deutsche Alphabet mit Leerstelle, Klammern, $' := '$ und $' ; '$
- $P := \{$
 $\langle \text{Loop-Programm}_n \rangle \rightarrow \varepsilon \mid \langle \text{Anweisungsblock} \rangle ; \langle \text{Loop-Programm}_n \rangle,$
 $\langle \text{Anweisungsblock} \rangle \rightarrow \langle \text{Elementaranweisung} \rangle \mid$
 $\text{loop } \langle \text{Variable}_n \rangle \text{ do } \langle \text{Loop-Programm}_n \rangle \text{ end},$
 $\langle \text{Elementaranweisung} \rangle \rightarrow \langle \text{Variable}_n \rangle := 0 \mid$
 $\langle \text{Variable}_n \rangle := \text{suc}(\langle \text{Variable}_n \rangle) \mid \langle \text{Variable}_n \rangle := \text{pred}(\langle \text{Variable}_n \rangle),$
 $\langle \text{Variable}_n \rangle \rightarrow x_0 \mid \dots \mid x_n \}$
- Das letzte Semikolon eines (Teil)-Programms wird üblicherweise nicht aufgeschrieben.
- P Loop-Programm $:\Leftrightarrow \exists n \in \mathbb{N}. P \in L(G_n)$

Semantik I

- Speichervektordarstellung

$$v = (\underbrace{x_0}_{\text{Ergebnis}}, \underbrace{x_1, \dots, x_n}_{\text{Eingaben}}, \underbrace{x_{n+1}, x_{n+2}, \dots}_{\text{Hilfsvariablen}})$$

- Ergebnis und Hilfsvariablen zu Beginn auf 0 gesetzt
- $v : \mathbb{N} \rightarrow \mathbb{N}$
- $v[y/x_i]$: Vektor v , bei dem y die Stelle x_i ersetzt
- $\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$

Semantik II

$$\delta : (\mathbb{N} \rightarrow \mathbb{N}) \times \mathcal{LP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$$

- Leeres Programm: $\delta(v, \varepsilon) := v$
- Wertzuweisungen:
 - $\delta(v, x_i := 0) := v[0/x_i]$
 - $\delta(v, x_i := \text{succ}(x_j)) := v[x_j + 1/x_i]$
 - $\delta(v, x_i := \text{pred}(x_j)) := \begin{cases} v[x_j - 1/x_i] & , \text{ falls } x_j \geq 1 \\ v[0/x_i] & , \text{ sonst} \end{cases}$
- Komposition: $\delta(v, P_1; P_2) := \delta(\delta(v, P_1), P_2)$,
wobei P_1 ein Anweisungsblock, d.h. eine Loop-Schleife oder eine Elementaranweisung, sein muss
- Loop-Schleife: $\delta(v, \text{loop } x_i \text{ do } P \text{ end}) := \delta(v, \underbrace{P; P; \dots; P}_{x_i \text{ Kopien}})$

Loop-Berechenbarkeit

- Sei $n \in \mathbb{N}$. $f : \mathbb{N}^n \rightarrow \mathbb{N}$ heißt Loop-berechenbar
 $: \iff \exists P \in \mathcal{LP}, \forall x_1, \dots, x_n \in \mathbb{N},$
 $\delta((0, x_1, \dots, x_n, 0, \dots), P)(0) = f(x_1, \dots, x_n)$
- $\mathcal{LF} :=$ Menge aller Loop-berechenbaren Funktionen

Addition I

- Eingaben x_1, x_2
- Programm add:
 $x_0 := x_1$; // Zuweisungsmakro, Definition später
 loop x_2 do
 $x_0 := \text{succ}(x_0)$
 end
- $\delta((0, 5, 2, 0, \dots), \text{add}) =$
 $\delta((0, 5, 2, 0, \dots), x_0 := x_1; \text{loop } x_2 \text{ do } x_0 := \text{succ}(x_0) \text{ end}) =$
 $\delta(\delta((0, 5, 2, 0, \dots), x_0 := x_1), \text{loop } x_2 \text{ do } x_0 := \text{succ}(x_0) \text{ end}) =$
 $\delta((5, 5, 2, 0, \dots), \text{loop } x_2 \text{ do } x_0 := \text{succ}(x_0) \text{ end}) =$
 $\delta((5, 5, 2, 0, \dots), x_0 := \text{succ}(x_0); x_0 := \text{succ}(x_0)) =$
 $\delta(\delta((5, 5, 2, 0, \dots), x_0 := \text{succ}(x_0)), x_0 := \text{succ}(x_0)) =$
 $\delta((6, 5, 2, 0, \dots), x_0 := \text{succ}(x_0)) =$
 $(7, 5, 2, 0, \dots)$

Addition II

- Allgemein:
 $\forall n, m \in \mathbb{N}, \delta((0, n, m, 0, \dots), \text{add}) = (n + m, n, m, 0, \dots)$
- $\forall n, m \in \mathbb{N}, \delta((0, n, m, 0, \dots), \text{add})(0) = n + m$
- $\text{add} \in \mathcal{LF}$

Die Konstantenfunktionen

- Programm const_n :

$$\underbrace{x_0 := \text{suc}(x_0); \dots; x_0 := \text{suc}(x_0)}_{n\text{-mal}}$$
- Programm unabhängig von Eingaben und deren Anzahl
- $\forall n, m \in \mathbb{N}, \text{const}_n^m \in \mathcal{LF}$, wobei
 $\text{const}_n^m : \mathbb{N}^m \rightarrow \mathbb{N}$ mit $\text{const}_n^m(x_0, \dots, x_{m-1}) = n$

Projektionen

- Programm pr_n :
 $x_0 := \text{suc } x_{n+1}$
 $x_0 := \text{pred } x_0$
- $\forall n, m \in \mathbb{N}, n < m \rightarrow \text{pr}_n^m \in \mathcal{LF}$, wobei
 $\text{pr}_n^m : \mathbb{N}^m \rightarrow \mathbb{N}$ mit $\text{pr}_n^m(x_0, \dots, x_{m-1}) = x_n$
- Achtung: Die Stelle x_n der Funktion ist die Eingabe x_{n+1} im Programm wegen der zusätzlichen Ausgabe x_0

Makros I

- Sei $N_P :=$ der größte im Programm P vorkommende Index und P_M das aktuelle Programm mit Makros
- $x_i := x_j$
 $x_i := \text{suc}(x_j);$
 $x_i := \text{pred}(x_i)$
- Für $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit $\text{op} \in \mathcal{LF}$ ($P_{\text{op}} \in \mathcal{LP}$):
 $x_i := \text{op}(x_{j_0}, \dots, x_{j_{n-1}})$, z. B. $x_3 := x_5 + x_2$
 $x_{N_{P_M}+2} := x_{j_0}; \dots; x_{N_{P_M}+n+1} := x_{j_{n-1}};$
 "Programm P'_{op} , bei dem gegenüber P_{op} alle Variablenindizes um $N_{P_M} + 1$ erhöht sind";
 $x_i := x_{N_{P_M}+1};$
 $x_{N_{P_M}+1} := 0; \dots; x_{\max(N_{P_M}+n+1, N_{P'_{\text{op}}})} := 0$

Makros II

- Für $0 \leq i < n$, $\text{op}_i : \mathbb{N}^{m_i} \rightarrow \mathbb{N}$ und $\text{op} : \mathbb{N}^n \rightarrow \mathbb{N}$ mit
 $\text{op}_i, \text{op} \in \mathcal{LF} (P_{\text{op}_i}, P_{\text{op}} \in \mathcal{LP})$:
 $x_i := \text{op} (\text{op}_0(x_{j_{00}}, \dots, x_{j_{0m_0-1}}), \dots, \text{op}_{n-1}(x_{j_{n-10}}, \dots, x_{j_{n-1m_{n-1}-1}}))$,
 z. B. $x_{73} := (x_{42} + x_{12}) * \text{suc } x_5$
 $x_{N_{P_M}+2} := \text{op}_0(x_{j_{00}}, \dots, x_{j_{0m_0-1}});$
 $\dots$
 $x_{N_{P_M}+n+1} := \text{op}_{n-1}(x_{j_{n-10}}, \dots, x_{j_{n-1m_{n-1}-1}});$
 $P'_{\text{op}};$
 $x_i := x_{N_{P_M}+1};$
 $x_{N_{P_M}+1} = 0; \dots; x_{\max(N_{P_M}+n+1, N_{P'_{\text{op}}})} := 0$

Makros III

- if $x_i = 0$ then P end
 - $x_{N_{P_M}+1} := 1;$
 - loop x_i do $x_{N_{P_M}+1} := 0$ end;
 - loop $x_{N_{P_M}+1}$ do P end;
 - $x_{N_{P_M}+1} := 0$
- if $x_i = n$ then P end
 - $x_{N_{P_M}+1} := (x_i - n) + (n - x_i);$ // Subtraktionsmakro in Übung
 - if $x_{N_{P_M}+1} = 0$ then P end;
 - $x_{N_{P_M}+1} := 0$
- ...

Vergleich zur Turing-Berechenbarkeit

- Alle $f \in \mathcal{LF}$ per Konstruktion total ($\mathcal{TLF} = \mathcal{LF}$)
- Unterschied $f : \mathbb{N}^n \rightarrow \mathbb{N}$ zu $f : \mathbb{N} \rightarrow \mathbb{N}$ essentiell oder nicht?
- $\mathcal{LF}$ vs $\mathcal{TLF}$:
 - $\mathcal{LF} \subseteq \mathcal{TLF}$?
 - $\mathcal{LF} \supseteq \mathcal{TLF}$?
 - $\mathcal{LF} = \mathcal{TLF}$?
 - Nichts von alledem?

Definition

- **While**-Programme bestehen aus folgenden Komponenten:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren `suc` und `pred`
 - dem Zuweisungsoperator `:=`
 - dem Kompositionsoperator `;`
 - dem Schleifenkonstrukt **while** do end
- **WP** := Menge aller **While**-Programme
- Die Syntax ließe sich wie bei den Loop-Programmen formal über eine kontextfreie Grammatik beschreiben

Semantik

- Analog zu Loop-Programmen bis auf wenige Änderungen
- $\delta :_{\subseteq} ((\mathbb{N} \rightarrow \mathbb{N}) \cup \{\perp\}) \times \mathcal{WP} \rightarrow (\mathbb{N} \rightarrow \mathbb{N})$
- $T_{v,P,i} := \{k \in \mathbb{N} \mid \delta(v, P^k)(i) = 0\}$
- $\delta(v, \text{while } x_i \text{ do } P \text{ end}) :=$

$$\begin{cases} \perp & , \text{ falls } T_{v,P,i} = \emptyset \\ \delta(v, P^{\min T_{v,P,i}}) & , \text{ falls } T_{v,P,i} \neq \emptyset \end{cases}$$
- $\delta(\perp, P) = \perp$

While-Berechenbarkeit

- Sei $n \in \mathbb{N}$. $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ heißt **While**-berechenbar

$$: \iff \exists P \in \mathcal{WP}, \forall x_1, \dots, x_n \in \mathbb{N},$$

$$(f(x_1, \dots, x_n) = \perp \rightarrow \delta((0, x_1, \dots, x_n, 0, \dots), P) = \perp) \wedge$$

$$(f(x_1, \dots, x_n) \neq \perp \rightarrow \delta((0, x_1, \dots, x_n, 0, \dots), P)(0) =$$

$$f(x_1, \dots, x_n))$$
- $\mathcal{WF} :=$ Menge aller **While**-berechenbaren Funktionen
- $\mathcal{TWf} :=$ Menge aller totalen **While**-berechenbaren Funktionen

Die konstant nicht definierte Funktion

- Programm $\text{const}_\perp$:
 $x_1 := \text{succ } x_2; \text{while } x_1 \text{ do } x_2 := 0 \text{ end}$
- Programm unabhängig von Eingaben und deren Anzahl
- $\forall m \in \mathbb{N}, \text{const}_\perp^m \in \mathcal{WF}$, wobei
 $\text{const}_\perp^m : \subseteq \mathbb{N}^m \rightarrow \mathbb{N}$ mit $\text{const}_\perp^m(x_0, \dots, x_{m-1}) = \perp$

Makros I

- loop x_i do P end

```

 $x_{N_{P_M}+1} := x_i;$ 
while  $x_{N_{P_M}+1}$  do
     $P;$ 
     $x_{N_{P_M}+1} := \text{pred}(x_{N_{P_M}+1})$ 
end
    
```

- while $x_i < x_j$ do P end

```

 $x_{N_{P_M}+1} := x_j - x_i$ 
while  $x_{N_{P_M}+1}$  do
     $P;$ 
     $x_{N_{P_M}+1} := x_j - x_i$ 
end
    
```


Makros II

- while $x_i > x_j$ do P end

$x_{N_{P_M}+1} := x_i - x_j$
while $x_{N_{P_M}+1}$ do
 P ;
 $x_{N_{P_M}+1} := x_i - x_j$
end

- while $x_i = x_j$ do P end

$x_{N_{P_M}+1} := 1 - ((x_i - x_j) + (x_j - x_i))$
while $x_{N_{P_M}+1}$ do
 P ;
 $x_{N_{P_M}+1} := 1 - ((x_i - x_j) + (x_j - x_i))$
end

Vergleich zur Loop-Berechenbarkeit

- Enthält nicht totale Funktionen ($\mathcal{TW}\mathcal{F} \subset \mathcal{WF}$)
- $\mathcal{LF} \subset \mathcal{WF}$
- Ist $\mathcal{TW}\mathcal{F} = \mathcal{LF}$?

Vergleich zur Turing-Berechenbarkeit

- $\mathcal{WF}$ vs $\mathcal{TF}$:
 - $\mathcal{WF} \subseteq \mathcal{TF}$?
 - $\mathcal{WF} \supseteq \mathcal{TF}$?
 - $\mathcal{WF} = \mathcal{TF}$?
 - Nichts von alledem?

Definition

- **Goto**-Programme als Sequenzen $M_1 : A_1; \dots; M_n : A_n$, wobei die A_i aus folgenden Komponenten bestehen:
 - der Konstanten 0
 - den Variablen x_i für $i \in \mathbb{N}$
 - den einstelligen Operatoren `suc` und `pred`
 - dem Zuweisungsoperator `:=`
 - dem bedingten Sprungbefehl `if  $x_i$  goto  $M_j$` für $j \neq 0$
 - dem Stoppbefehl **halt** (Vereinbarung: Am Ende eines jeden Programms stehe implizit ebenfalls $M_{n+1} : \text{halt}$, um ein Rauslaufen zu vermeiden (s.u.))
- **GP** := Menge aller **Goto**-Programme
- Formale Definition wie bei Loop-Programmen über kontextfreie Grammatiken, wobei in diesem Fall der Umgang mit den Markern bedacht werden muss

Semantik

- Für jedes $P \in \mathcal{GP}$ sei $\vdash_P \subseteq ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N}) \times ((\mathbb{N} \rightarrow \mathbb{N}) \times \mathbb{N})$ wie folgt definiert
- Kommt in P die Zeile $M_k : A_k$ mit $A_k \in \{x_i := 0, x_i := \text{suc}(x_j), x_i := \text{pred}(x_j)\}$ vor, so $(v, k) \vdash_P (\delta_{\text{Loop}}(v, A_k), k + 1)$
- Kommt in P die Zeile $M_k : \text{if } x_i \text{ goto } M_l$ vor, so $(v, k) \vdash_P \begin{cases} (v, l) & , \text{ falls } x_i \neq 0 \\ (v, k + 1) & , \text{ falls } x_i = 0 \end{cases}$
- Kommt in P die Zeile $M_k : \text{halt vor}$, so $(v, k) \vdash_P (v, 0)$
- Reflexiv-transitiver Abschluss $\vdash_P^*$

Goto-Berechenbarkeit

- Sei $n \in \mathbb{N}$. $f : \subseteq \mathbb{N}^n \rightarrow \mathbb{N}$ heißt **Goto**-berechenbar
 $: \iff \exists P \in \mathcal{GP}, \forall x_1, \dots, x_n \in \mathbb{N},$
 $(f(x_1, \dots, x_n) = \perp \rightarrow \neg \exists v : \mathbb{N} \rightarrow \mathbb{N},$
 $((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0))$
 $\wedge$
 $(f(x_1, \dots, x_n) \neq \perp \rightarrow \exists v : \mathbb{N} \rightarrow \mathbb{N},$
 $((0, x_1, \dots, x_n, 0, \dots), 1) \vdash_P^* (v, 0) \wedge v(0) = f(x_1, \dots, x_n))$
- $\mathcal{GF} :=$ Menge aller **Goto**-berechenbaren Funktionen
- $\mathcal{TGF} :=$ Menge aller totalen **Goto**-berechenbaren Funktionen

Addition I

- Programm add:

$M_1 : x_0 := x_2;$

$M_2 : \text{if } x_1 \text{ goto } M_4;$

$M_3 : \text{halt};$

$M_4 : x_0 := \text{succ}(x_0);$

$M_5 : x_1 := \text{pred}(x_1);$

$M_6 : \text{if } x_0 \text{ goto } M_2$

- $((0, 3, 4, 0, \dots), 1) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 2) \vdash_{\text{add}} ((4, 3, 4, 0, \dots), 4) \vdash_{\text{add}}$
 $((5, 3, 4, 0, \dots), 5) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 6) \vdash_{\text{add}} ((5, 2, 4, 0, \dots), 2) \vdash_{\text{add}}$
 $((5, 2, 4, 0, \dots), 4) \vdash_{\text{add}} ((6, 2, 4, 0, \dots), 5) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 6) \vdash_{\text{add}}$
 $((6, 1, 4, 0, \dots), 2) \vdash_{\text{add}} ((6, 1, 4, 0, \dots), 4) \vdash_{\text{add}} ((7, 1, 4, 0, \dots), 5) \vdash_{\text{add}}$
 $((7, 0, 4, 0, \dots), 6) \vdash_{\text{add}} ((7, 0, 4, 0, \dots), 2) \vdash_{\text{add}} ((7, 0, 4, 0, \dots), 3) \vdash_{\text{add}}$
 $((7, 0, 4, 0, \dots), 0)$

Addition II

- Allgemein: $((0, n, m, 0, \dots), 1) \vdash_{\text{add}}^* ((n + m, 0, m, 0, \dots), 0)$
- $\text{add} \in \mathcal{TGF}$

Makros (Vorbetrachtungen)

- Bisher Abkürzungen ohne Nebenwirkungen
z. B. ändert $x_3 := (x_1 + x_5) * x_7$ nicht einfach x_{12}
- Jetzt (wegen Nicht-Linearität) Erweiterungen am System, deren jeweilige Äquivalenz zum Originalmodell eigentlich nachgewiesen werden müsste
- Geben nur Skizzen an, die die Simulation grob aufzeigen
- Variablen, die in Makro nicht auftauchen, neu

Makros

- goto M_k
 $x_j := 1;$
 if x_j goto M_k
- if $x < y$ goto M_k
 $z := y - x;$
 if z goto M_k
- if $x > y$ goto M_k
 $z := x - y;$
 if z goto M_k
- if $x = y$ goto M_k
 $z := (y - x) + (x - y);$
 if $z < 1$ goto M_k

Vergleich zur While-Berechenbarkeit I

Theorem

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:
 - Sei $f \in \mathcal{WF}$
 - Gibt $P_W \in \mathcal{WP}$, das diese Funktion berechnet
 - Sei $P_W = A_1; \dots; A_n$
 - Konstruiere zunächst $M_1 : A_1; \dots; M_n : A_n; M_{n+1} : \text{halt}$
 - Ersetze jedes $M_i : \text{while } x_i \text{ do } P \text{ end}$ durch
 $M_i : \text{if } x_i = 0 \text{ goto } M_{i+1};$
 $P;$
 $M_{\text{new}_i} : \text{goto } M_i;$
 $M_{i+1} : \dots$

Vergleich zur While-Berechenbarkeit II

Theorem

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\supseteq$ ”:
 - ...
 - Übersetze P rekursiv durch ein äquivalentes $P' \in \mathcal{GP}$, aber lasse dabei den letzten halt-Befehl weg
 - Baue fehlende Marker ein, shifte die Nummerierung der alten Marker entsprechend und passe auch die goto-Befehle an, um P_G zu erhalten
 - Weise nach, dass $f_{P_G} = f_{P_W} = f$

Vergleich zur While-Berechenbarkeit III

Theorem

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”:
 - Sei $f \in \mathcal{GF}$
 - Gibt $P_G \in \mathcal{GP}$, das diese Funktion berechnet
 - Sei $P_G = M_1 : A_1; \dots; M_n : A_n$
 - Konstruktion von P_W mit Extra-Variable y , um Marker zu simulieren, sowie einer (sichtbaren) While-Schleife zur Fallunterscheidung
- ...

Vergleich zur While-Berechenbarkeit IV

Theorem

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”: ...
 $y := 1;$
 while $y \neq 0$ do
 if $y = 1$ then
 ... // A_1
 end;
 ...
 if $y = n$ then
 ... // A_n
 end
 end

Vergleich zur While-Berechenbarkeit V

Theorem

$$\mathcal{GF} = \mathcal{WF}$$

Beweisskizze

- “ $\subseteq$ ”:
- ...
- Übersetze $M_k : A_k$:
 - $M_k : x_i := 0$ durch $x_i := 0; y := k + 1$
 - $M_k : x_i := \text{succ}(x_j)$ durch $x_i := \text{succ}(x_j); y := k + 1$
 - $M_k : x_i := \text{pred}(x_j)$ durch $x_i := \text{pred}(x_j); y := k + 1$
 - $M_k : \text{if } x_i \text{ goto } M_l$ durch $y := k + 1; \text{if } x_i \text{ then } y := l \text{ end}$
 - $M_k : \text{halt}$ durch $y := 0$
- Weise nach, dass $f_{P_W} = f_{P_G} = f$

Nebenergebnis und Korollar

Theorem (Satz von Kleene)

Jede While-berechenbare Funktion lässt sich – abgesehen von Loop-Simulationen – durch ein While-Programm mit nur einer While-Schleife berechnen

Korollar

$$\mathcal{TGF} = \mathcal{TWF}$$

Vergleich zur Turing-Berechenbarkeit I

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:
 - Sei $f \in \mathcal{WF}$
 - Dann gibt es P_W , das dieses berechnet
 - Wandle dieses gemäß der Konstruktion des letzten Satzes zunächst in P_G und dann in P'_W um, wobei letzteres Programm die folgende Gestalt hat (A_i ohne echte While-Schleifen): ...

Vergleich zur Turing-Berechenbarkeit II

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- “ $\subseteq$ ”: ...
 $y := 1;$
 while $y \neq 0$ do
 if $y = 1$ then
 ... // A_1
 end;
 ...
 if $y = n$ then
 ... // A_n
 end
 end

Vergleich zur Turing-Berechenbarkeit III

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$: ...
 - Simulation mit TM M mit $N_{P'_W} + 1$ Bändern: eins für jede Variable und ein Band zum Herunterzählen für Loop-Schleifen
 - Konstruktion von Elementarmaschinen zum Nullsetzen ($\text{Init}(x_i)$), Kopieren ($\text{Copy}(x_i, x_j)$), bilden des Vorgängers ($\text{Pred}(x_i)$) und des Nachfolgers ($\text{Suc}(x_i)$)
 - Aufbauend darauf lassen sich TMs konstruieren für die Hintereinanderausführung sowie Loop-Schleifen (mittels des Hilfsbandes)
 - D.h. für jedes A_i lässt sich eine Maschine M_i konstruieren

Vergleich zur Turing-Berechenbarkeit IV

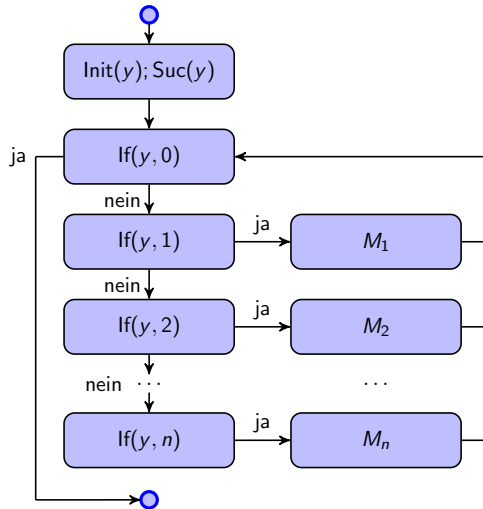
Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{WF} \subseteq \mathcal{TF}$:
 - ...
 - Weiterhin lässt sich für jedes n eine Maschine $lf(y, n)$ mit einem Ja- und einem Nein-Ausgang konstruieren, die checkt, ob $y = n$ ist
 - Es lässt sich nun M wie auf der nächsten Folie dargestellt konstruieren
 - Es verbleibt dann eigentlich noch zu zeigen, dass
$$f_M = f_{P'_W} = f_{P_W} = f$$

Vergleich zur Turing-Berechenbarkeit V



Vergleich zur Turing-Berechenbarkeit VI

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:
 - Sei $f \in \mathcal{TF}$
 - Es gibt dann eine zugehörige unäre Funktion $f' :_{\subseteq} \{1\}^* \rightarrow \{1\}^*$ mit $f' \in \mathcal{TF}_{\{1\}}$
 - Und damit wiederum eine TM $T = (Q, \{1\}, \Gamma, \delta, q_0, B, F)$, die f' berechnet
 - O.B.d.A. seien $Q = \{0, \dots, n-1\}$, $\Gamma = \{0, \dots, m-1\}$ ($m \geq 2$, da $1, B \in \Gamma$) und $B = 0$
 - Grundidee für Konstruktion von P_G : Codiere Konfigurationen und simulierte die Konfigurationsübergänge

Vergleich zur Turing-Berechenbarkeit VII

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:
 - ...
 - Konfiguration (α, q, β) bei $\alpha = a_{\ell-1} \dots a_0$ und $\beta = b_0 \dots b_{k-1}$ durch
 - $x_q := q$
 - $x_\alpha := \sum_{i=0}^{\ell-1} a_i * m^i$
 - $x_\beta := \sum_{i=0}^{k-1} b_i * m^i$
 - x_α und x_β m -adische Darstellungen (wegen $m \geq 2$ eindeutig)

Vergleich zur Turing-Berechenbarkeit VIII

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:
 - ...
 - Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$
 - $\delta(i, j) = (i', j', L)$
$$\begin{aligned}x_\beta &:= x_\beta \text{ div } m; \\x_\beta &:= j' + m * x_\beta; \\x_\beta &:= x_\alpha \text{ mod } m + m * x_\beta; \\x_\alpha &:= x_\alpha \text{ div } m; \\x_q &:= i'\end{aligned}$$

Vergleich zur Turing-Berechenbarkeit IX

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$:
 - Konstruiere $P_{G_{ij}}$ für alle i, j mit $\delta(i, j) \neq \perp$
 - ...
 - $\delta(i, j) = (i', j', R)$

$$x_\beta := x_\beta \text{ div } m;$$

$$x_\beta := j' + m * x_\beta;$$

$$x_\alpha := x_\beta \text{ mod } m + m * x_\alpha;$$

$$x_\beta := x_\beta \text{ div } m;$$

$$x_q := i'$$

Vergleich zur Turing-Berechenbarkeit X

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - Konstruiere Hauptteil von P_G , der Konfigurationsübergänge simuliert, wie folgt (nur Paare für $\delta(i, j) \neq \perp$):
 $M_2 : x_a := x_\beta \text{ mod } m;$
 if $(x_q = 1 \wedge x_a = 1)$ goto $M_{11};$
 if $(x_q = 1 \wedge x_a = 2)$ goto $M_{12};$
 ...
 if $(x_q = 2 \wedge x_a = 1)$ goto $M_{21};$
 if $(x_q = 2 \wedge x_a = 3)$ goto $M_{22};$ // Bsp. $\delta(2, 2) = \perp$
 ...
 if $x_q \in F \wedge x_\alpha = 0$ goto M_3 else goto $M_2;$ // $\alpha = B^\ell$

Vergleich zur Turing-Berechenbarkeit XI

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - Konstruiere Hauptteil des Programms P_G , der Konfigurationsübergänge simuliert, wie folgt:

...

$M_{11} : P_{G_{11}}; \text{ goto } M_2;$

$M_{12} : P_{G_{12}}; \text{ goto } M_2;$

...

$M_{21} : P_{G_{21}}; \text{ goto } M_2;$

$M_{23} : P_{G_{23}}; \text{ goto } M_2;$

...

...

Vergleich zur Turing-Berechenbarkeit XII

Theorem

$$\mathcal{GF} = \mathcal{WF} = \mathcal{TF}$$

Beweisskizze

- $\mathcal{TF} \subseteq \mathcal{GF}$: ...
 - P_G muss zunächst Variablen initialisieren
 - $M_1 : x_\alpha := 0;$
 - $x_q := q_0;$
 - $x_\beta := \sum_{i=0}^{\text{Eingabe}} m^i$
 - Dann obiger Hauptteil beginnend mit Marker M_2
 - Schließlich über Marker M_3 Decodierung von x_β und Testen, ob alle b_i – wie gefordert – 1en sind bis nur noch Blanks (0) kommen
 - Eigentlich noch Nachweis der Korrektheit der Konstruktion

Übertragung auf totale Berechenbarkeitsmodelle

Korollar

$$TGF = TWF = TTF$$

Zusammenfassung

- $\mathcal{LF} \subseteq \mathcal{TGF} = \mathcal{TWf} = \mathcal{TTf} \subset \mathcal{GF} = \mathcal{Wf} = \mathcal{TF}$
- $\mathcal{TGF} = \mathcal{TWf} = \mathcal{TTf} \subseteq \mathcal{LF}$?
- Mit Berechenbarkeitsmodellen genau die (total) berechenbaren Funktionen akkurat getroffen?